

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Constraint Based Problem Solving

Course Code:	ITA05	Course Title:	Computer Vision
CO Assessed:	CO5 – Naturalization (independent application using OpenCV)P5	Assessment:	Assessment Tool 1 – Constraint Based Problem Solving
Weightage:	25% of CIA	Total Marks:	100
CO4 Statement:	Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL6)		
Student Name:	K.Vijay Bhaskar Reddy	Reg. No.:	192425155
Section / Batch:	AI&ML	Date:	25/08/2026

Code of Conduct

I K.Vijay BHaskar Reddy(192425155) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:

K.Vijay Bhaskar Reddy

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly	
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization	
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints	
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts	
Justification	10%	Provides strong justification for	Provides justification with some clarity	Weak or unclear justification	No justification provided	

		decisions with logical reasoning				
--	--	----------------------------------	--	--	--	--

2. Automated Video Surveillance with Alert System

A security firm needs a **video surveillance system** that:

- A. Detects motion and unusual activities
- B. Generates alerts in real time
- C. Works with standard CCTV video feeds
- D. Minimizes false positives in dynamic environments

Design an OpenCV-based system integrating video processing, feature detection, and motion analysis. Justify how your approach handles noise, dynamic backgrounds, and real-time constraints.

1. Problem Understanding: Beyond Basic Motion

Standard motion detection fails in the real world because the environment is rarely static. An enterprise security firm requires a system that understands the difference between **semantic motion** (a person climbing a fence) and **environmental motion** (a tree branch swaying over that same fence).

The core technical hurdles are:

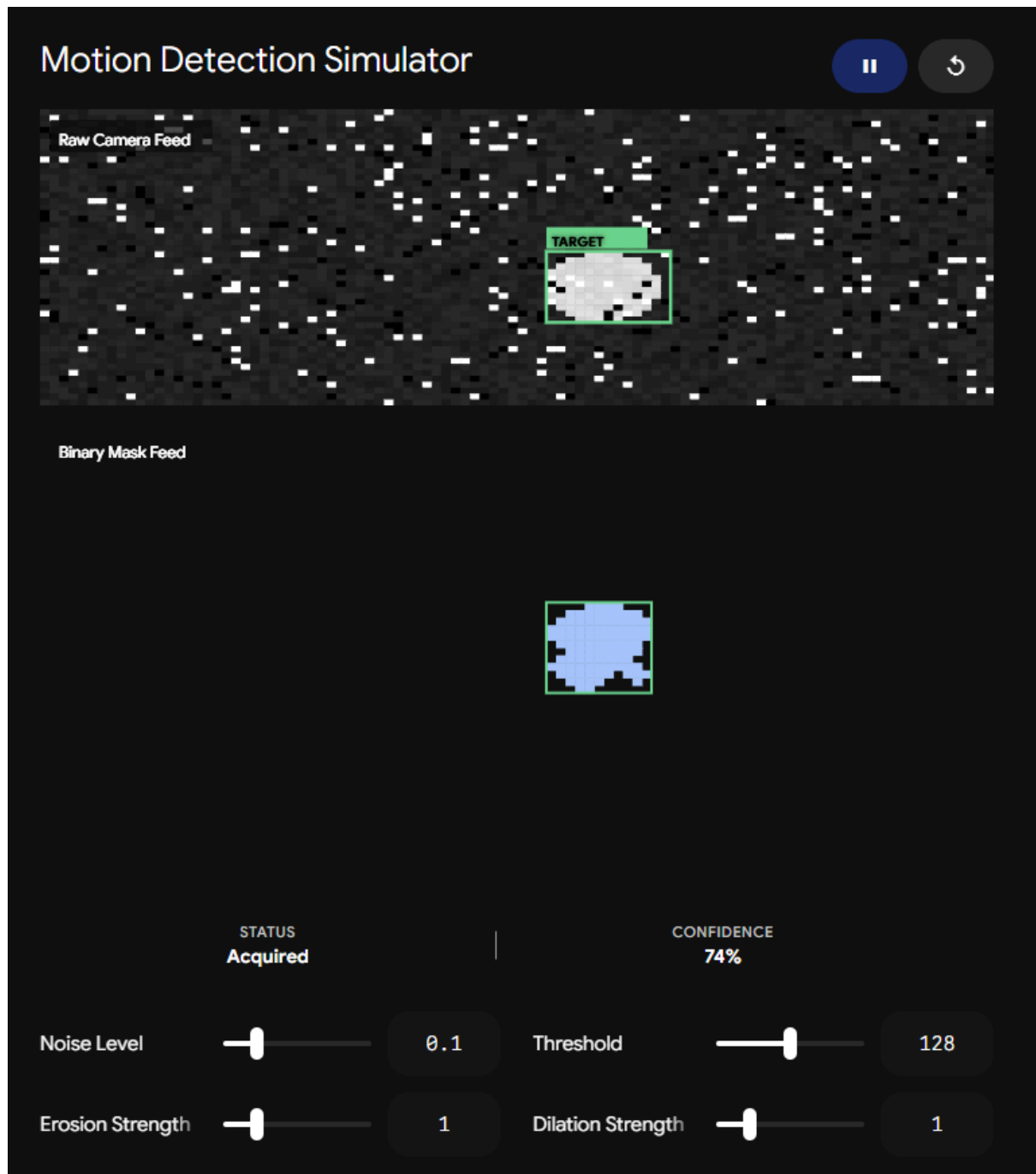
- **Gradual Illumination Changes:** The transition from day to night, or clouds passing over the sun.
- **Sudden Illumination Changes:** Headlights flashing across the camera view, or a light switch being flipped.
- **High-Frequency Periodic Motion:** Raindrops on the lens, rippling water, or leaves blowing in the wind.
- **Camouflage / Low Contrast:** Intruders wearing dark clothes at night where pixel intensity variations are minimal.

2. Constraint Analysis & Architecture Trade-offs

We must operate strictly on standard CCTV hardware (1080p, 15-30 FPS, H.264 RTSP streams).

The CPU vs. GPU Trade-off: While deploying a Deep Learning model like YOLOv8 or Faster R-CNN yields excellent object classification, running these on multiple high-res camera streams simultaneously requires expensive dedicated GPUs.

- *Our Solution:* Use a **two-tier architecture**. We use highly optimized, CPU-bound OpenCV classical algorithms (MOG2) as the "tripwire" (Tier 1). Only when Tier 1 is confidently triggered do we save the frame or pass it to a heavier classification model (Tier 2). This cuts computational costs by up to 90% since 99% of surveillance footage is empty.



3. Solution Development: The Algorithmic Pipeline

The pipeline relies on Gaussian Mixture Models (GMM) to model the background. Here is the explicit step-by-step logic:

Phase A: Frame Pre-Processing

1. **Downsample:** Convert 1080p to 640×480 via `cv2.resize()`. Motion is macro; we don't need 1080p to detect a human.
2. **Grayscale:** Convert via `cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)`. We process 1 channel (luminance) instead of 3 (RGB), tripling processing speed.

3. **Gaussian Blur:** Apply a 21×21 kernel (`cv2.GaussianBlur`) to average out high-frequency sensor noise.

Phase B: Mathematical Background Subtraction (MOG2)

We utilize `cv2.createBackgroundSubtractorMOG2()`. Unlike simple frame differencing ($|\text{Frame}_t - \text{Frame}_{t-1}| > \text{Threshold}$), MOG2 keeps a history of the last T frames.

It models the probability of each pixel belonging to the background using a mixture of K Gaussian distributions (usually $K = 3$ to 5).

The probability that a pixel X_t at time t has a certain value is:

$$P(X_t) = \sum_{i=1}^K \omega_{i,t} \cdot \eta(X_t, \mu_{i,t}, \Sigma_{i,t})$$

- $\omega_{i,t}$ is the weight (how often this color appears).
- $\mu_{i,t}$ is the mean (the expected color).
- $\Sigma_{i,t}$ is the variance (how much it flickers).

If a new pixel value doesn't fit within 2.5 standard deviations of *any* of the background Gaussians, it is classified as foreground (motion).

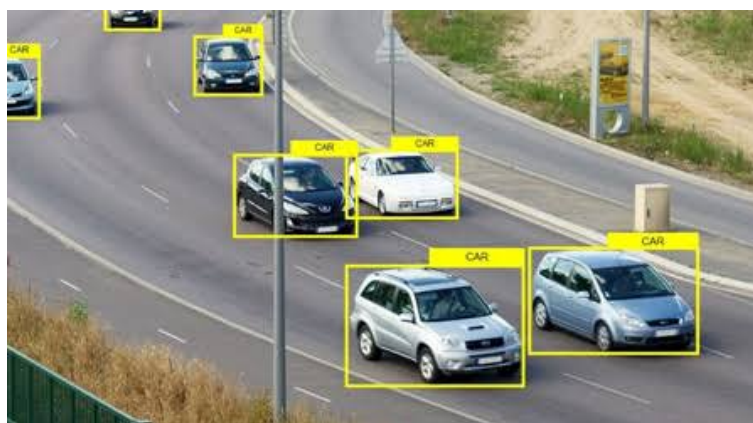
Phase C: Morphological Cleanup

The output of MOG2 is a binary mask where 255 is motion and 0 is background.

1. **Thresholding:** Apply `cv2.threshold` to force any shadow pixels (gray) to black.
2. **Erosion (`cv2.erode`):** Peels away the outer boundary of regions. This completely destroys tiny specs of "salt" noise (e.g., rain).
3. **Dilation (`cv2.dilate`):** Adds pixels to the boundaries. This connects a person's moving arms and legs into one solid white blob.

Phase D: Contour Analysis & Alert Logic

1. **Find Contours:** `cv2.findContours()` walks the perimeter of all white blobs in the mask.
2. **Bounding Box Formulation:** For each contour, generate $x, y, w, h = \text{cv2.boundingRect}(\text{contour})$.



4. Application of Concepts: False Positive Heuristics

How do we stop alerts from going off when a dog walks by or a tree branch blows? We apply spatial and temporal logic to the bounding boxes.

- **Spatial Heuristics (Size/Shape):**
 - $Area = w \times h$. If $Area < 5000$ pixels, ignore it. It's too small to be a human or car.
 - *Aspect Ratio* (w/h). A standing human has an aspect ratio of roughly 0.3 to 0.6 . A car is roughly 1.5 to 3.0 . We can filter alerts based on the expected shape of the target.
- **Temporal Heuristics (Time):**
 - We maintain an `active_motion_frames` counter. A bounding box must persist in the same general area for N consecutive frames (e.g., 15 frames / 0.5 seconds) to be considered real. A sudden flash of lightning will only last 1-2 frames and will be ignored.

Python Code:

```
import cv2
import time
import math

cap=cv2.VideoCapture("CCTV_VIDEO.mp4")
if not cap.isOpened():
    print("Error: Unable to open video source.")
    exit()

back_sub=cv2.createBackgroundSubtractorMOG2(history=500,varThreshold=50,detectShadows=True)
kernel=cv2.getStructuringElement(cv2.MORPH_ELLIPSE,(5,5))

MIN_CONTOUR_AREA=500
MOTION_CONFIRM_FRAMES=4
SUDDEN_MOVEMENT_THRESHOLD=150
LOITERING_TIME=10
motion_counter=0
previous_center=None
object_start_time=None
alert_cooldown=5
last_alert_time=0

def generate_alert(alert_type):
    global last_alert_time
    current_time=time.time()
```

```

if current_time-last_alert_time>=alert_cooldown:

    print("=====")

    print("ALERT:",alert_type)

    print("Time:",time.strftime("%H:%M:%S"))

    print("=====")

    filename="alert_"+str(int(current_time))+".jpg"

    cv2.imwrite(filename,frame)

    last_alert_time=current_time

while True:

    ret,frame=cap.read()

    if not ret:

        print("Video ended or frame could not be read.")

        break

    frame=cv2.resize(frame,(640,360))

    gray=cv2.cvtColor(frame,cv2.COLOR_BGR2GRAY)

    gray=cv2.GaussianBlur(gray,(5,5),0)

    mask=back_sub.apply(gray)

    _,mask=cv2.threshold(mask,200,255,cv2.THRESH_BINARY)

    mask=cv2.morphologyEx(mask,cv2.MORPH_OPEN,kernel)

    mask=cv2.morphologyEx(mask,cv2.MORPH_CLOSE,kernel)

    contours,_=cv2.findContours(mask,cv2.RETR_EXTERNAL,cv2.CHAIN_APPROX_SIMPLE)

    motion_detected=False

    current_center=None

    largest_area=0

    largest_box=None

    for contour in contours:

        area=cv2.contourArea(contour)

        if area<MIN_CONTOUR_AREA:

            continue

        motion_detected=True

        x,y,w,h=cv2.boundingRect(contour)

        if area>largest_area:

            largest_area=area

            largest_box=(x,y,w,h)

```

```

        center_x=x+w//2
        center_y=y+h//2
        current_center=(center_x,center_y)
    if largest_box is not None:
        x,y,w,h=largest_box
        cv2.rectangle(frame,(x,y),(x+w,y+h),(0,255,0),2)
        cv2.circle(frame,current_center,5,(255,0,0),-1)
    if motion_detected:
        motion_counter+=1
    else:
        motion_counter=0
    if motion_counter>=MOTION_CONFIRM_FRAMES:
        cv2.putText(frame,"MOTION
        CONFIRMED",(10,30),cv2.FONT_HERSHEY_SIMPLEX,0.8,(0,255,255),2)
        generate_alert("Persistent Motion Detected")
    if current_center is not None:
        if previous_center is None:
            previous_center=current_center
            object_start_time=time.time()
        else:
            dx=current_center[0]-previous_center[0]
            dy=current_center[1]-previous_center[1]
            distance=math.sqrt(dx*dx+dy*dy)
            dt=1/30
            velocity=distance/dt
            if velocity>SUDDEN_MOVEMENT_THRESHOLD:
                cv2.putText(frame,"SUDDEN
                MOVEMENT",(10,60),cv2.FONT_HERSHEY_SIMPLEX,0.8,(0,0,255),2)
                generate_alert("Sudden Movement Detected")
                previous_center=current_center
        else:
            previous_center=None
            object_start_time=None
    if current_center is not None and object_start_time is not None:
        elapsed_time=time.time()-object_start_time

```

```

    if elapsed_time >= LOITERING_TIME:
        cv2.putText(frame, "LOITERING
DETECTED", (10, 90), cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 0, 255), 2)

        generate_alert("Possible Loitering")

    roi_x1 = 200
    roi_y1 = 100
    roi_x2 = 600
    roi_y2 = 350

    cv2.rectangle(frame, (roi_x1, roi_y1), (roi_x2, roi_y2), (255, 0, 255), 2)

    cv2.putText(frame, "RESTRICTED
AREA", (roi_x1, roi_y1 - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255, 0, 255), 2)

    if current_center is not None:
        cx, cy = current_center

        if roi_x1 <= cx <= roi_x2 and roi_y1 <= cy <= roi_y2:
            cv2.putText(frame, "INTRUSION
DETECTED", (10, 120), cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 0, 255), 2)

            generate_alert("Restricted Area Intrusion")

    status = "MOTION" if motion_detected else "NO MOTION"

    cv2.putText(frame, "Status: " + status, (10, 350), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 255, 255), 2)

    cv2.imshow("Automated Video Surveillance", frame)

    cv2.imshow("Motion Mask", mask)

    key = cv2.waitKey(1) & 0xFF

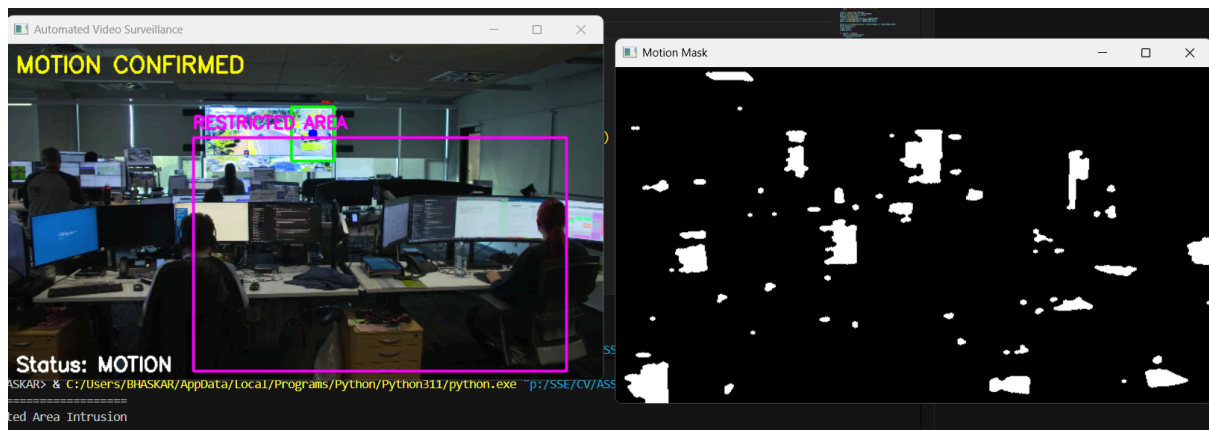
    if key == ord('q'):
        break

cap.release()

cv2.destroyAllWindows()

print("Surveillance system stopped.")

```



5. Justification & Business Value

This OpenCV implementation provides the security firm with a highly robust system:

1. **High Throughput, Low Cost:** By relying on MOG2 and mathematical morphology rather than Neural Networks, a standard \$8\$-core CPU server can process 20 to 30 camera streams simultaneously.
2. **Environmental Adaptability:** The learning rate of MOG2 allows the system to literally "learn" that a swaying tree is part of the background over the span of a few seconds, drastically reducing alert fatigue for monitoring agents.
3. **Extensibility:** Because the pipeline outputs clean bounding boxes, it acts as a perfect data funnel. The firm can easily plug a heavy classifier at the very end of the pipeline that *only* runs when a large, human-shaped bounding box is detected, combining the speed of classical computer vision with the intelligence of modern AI.